Assignment – 2

ID5130: Parallel Scientific Computing

Indian Institute of Technology Madras

Shubham Jadhav

Roll No: CS24S009

Date: March 2026

Contents

1 Question 1: 1D Traveling Wave Equation (40 points) 2
1.1 Problem Statement . . . 2
1.2 Discretization and Update Formulas . . . 2
1.3 Part (a): Serial Program . . . 3
1.3.1 Approach . . . 3
1.3.2 Terminal Output . . . 3
1.3.3 Plots . . . 3
1.4 Part (b): MPI Program . . . 4
1.4.1 Approach . . . 4
1.4.2 Verification . . . 4
1.4.3 Terminal Output . . . 4
1.4.4 Plots — $p = 2$ . . . 5
1.4.5 Plots — $p = 4$ . . . 5
1.5 Part (c): Comparison of Upwind and QUICK Schemes . . . 6
1.5.1 Zoomed Comparison at $t = 1.0$ . . . 6
1.5.2 Analysis . . . 6

2 Question 2: Poisson Equation — Jacobi Method (60 points) 7
2.1 Problem Statement . . . 7
2.2 Discretization . . . 7
2.3 Part (a): Serial Jacobi Program ($\Delta = 0.1$) . . . 8
2.3.1 Approach . . . 8
2.3.2 Terminal Output . . . 8
2.3.3 Results . . . 8
2.3.4 Plots . . . 9
2.3.5 Key Findings . . . 9
2.4 Part (b): MPI Jacobi Program ($\Delta = 0.01$) . . . 10
2.4.1 Approach . . . 10
2.4.2 Terminal Output . . . 10
2.4.3 Results . . . 11
2.4.4 Plots . . . 11
2.4.5 Key Findings . . . 11

3 Reference Material Used 12

Question 1: 1D Traveling Wave Equation (40 points)

Problem Statement

Consider the one-dimensional first-order traveling wave equation:

$$\frac{\partial u}{\partial t} + c\,\frac{\partial u}{\partial x} = 0 \tag{1}$$

with $c = 1.0$ in a domain of length $L = 2.0$. The initial condition is:

$$u(x, t = 0) = u_0(x) = \begin{cases} \sin(4\pi x), & 0 \le x \le 0.5 \\ 0, & 0.5 < x \le 2.0 \end{cases} \tag{2}$$

The exact solution at any time t is $u(x, t) = u_0(x - ct)$. The boundary conditions are $u(0, t) = 0$ and $u(L, t) = 0$.

The equation is solved using Euler explicit time integration with two spatial discretization schemes:

(i) First-order Upwind:

$$\frac{\partial u}{\partial x} \approx \frac{u_i^n - u_{i-1}^n}{\Delta x} \tag{3}$$

(ii) Third-order QUICK:

$$\frac{\partial u}{\partial x} \approx \frac{1}{\Delta x}\left(\frac{3}{8}u_i^n - \frac{7}{8}u_{i-1}^n + \frac{1}{8}u_{i-2}^n + \frac{3}{8}u_{i+1}^n\right) \tag{4}$$

Parameters: $\Delta x = 0.002$, $\Delta t = 0.0001$, giving CFL $= c\Delta t/\Delta x = 0.05$.

Discretization and Update Formulas

Substituting the spatial derivatives into the Euler explicit time-stepping formula $u_i^{n+1} = u_i^n - \Delta t \cdot c \cdot (\partial u/\partial x)_i$, the update equations become:

Upwind:

$$u_i^{n+1} = u_i^n - \mathrm{CFL} \cdot \left(u_i^n - u_{i-1}^n\right) \tag{5}$$

QUICK:

$$u_i^{n+1} = u_i^n - \mathrm{CFL} \cdot \left(\frac{3}{8}u_i^n - \frac{7}{8}u_{i-1}^n + \frac{1}{8}u_{i-2}^n + \frac{3}{8}u_{i+1}^n\right) \tag{6}$$

The upwind scheme uses a 2-point stencil $(i-1, i)$, while QUICK uses a 4-point stencil $(i-2, i-1, i, i+1)$. At the near-boundary point $i = 1$ (i.e., $x = \Delta x$), QUICK cannot be applied since it requires $u_{i-2} = u_{-1}$, so the upwind scheme is used there, as specified in the problem statement.

Stability: Both schemes are stable for CFL ≤ 1. With CFL $= 0.05$, the computation is well within the stability limit.

The grid has $N = L/\Delta x = 1000$ intervals and $N + 1 = 1001$ grid points. Time-stepping to $t = 0.5$ requires 5000 steps, and to $t = 1.0$ requires 10000 steps.

Part (a): Serial Program

1.3.1 Approach

The serial program implements both schemes in a straightforward loop over all grid points at each time step. The solution array `u[0..N]` is updated to `u_new[0..N]` each step, with boundary conditions enforced at $i = 0$ and $i = N$. The solution is written to CSV files at $t = 0$, 0.5, and 1.0 for each scheme.

1.3.2 Terminal Output

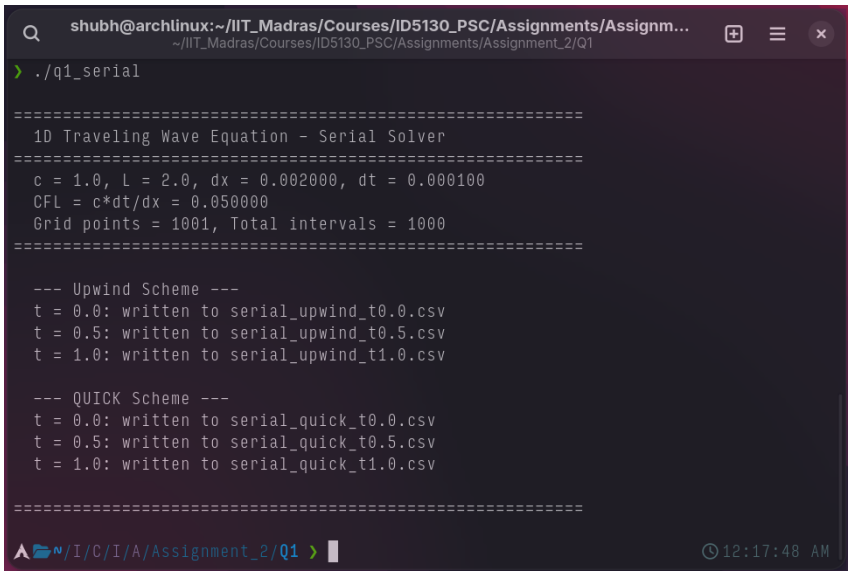
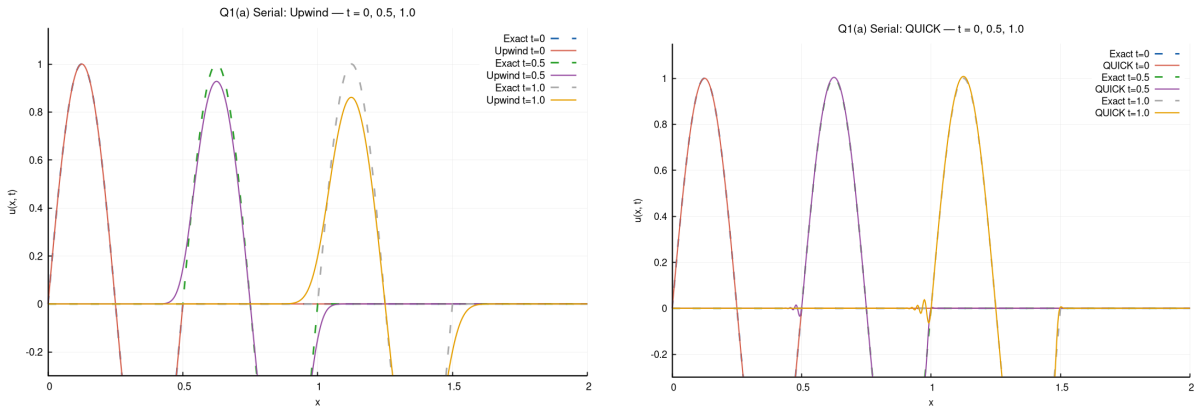


Figure 1: Q1(a): Terminal output of serial solver.

1.3.3 Plots



(a) Upwind scheme (b) QUICK scheme

Figure 2: Comparison of numerical and exact solutions at $t = 0$, 0.5, 1.0.

Part (b): MPI Program

1.4.1 Approach

The domain $[0, L]$ is decomposed into contiguous blocks along the x-direction. Each MPI process owns `local_n` consecutive grid points. Since the QUICK stencil reaches $i-2$ and $i+1$, each subdomain requires **2 left halo points and 1 right halo point**.

At every time step, the halo exchange proceeds as follows:

1. Each process sends its rightmost 2 data points to its right neighbor and receives 2 points from its left neighbor into its left halo. This fills the ghost cells needed for the u_{i-2} and u_{i-1} terms.

2. Each process sends its leftmost data point to its left neighbor and receives 1 point from its right neighbor into its right halo. This fills the ghost cell for the u_{i+1} term.

3. Processes at domain boundaries apply boundary conditions ($u = 0$) to their halo cells.

This communication pattern uses `MPI_Sendrecv` for deadlock-free bidirectional exchange.

After time-stepping, the full solution is gathered on rank 0 using `MPI_Gatherv` for output.

1.4.2 Verification

Serial and MPI solutions (for both $p = 2$ and $p = 4$) were compared at $t = 0$, 0.5, and 1.0 for both schemes. The maximum difference in all cases is **exactly 0.0**, confirming correct halo exchange and identical numerical results.

1.4.3 Terminal Output

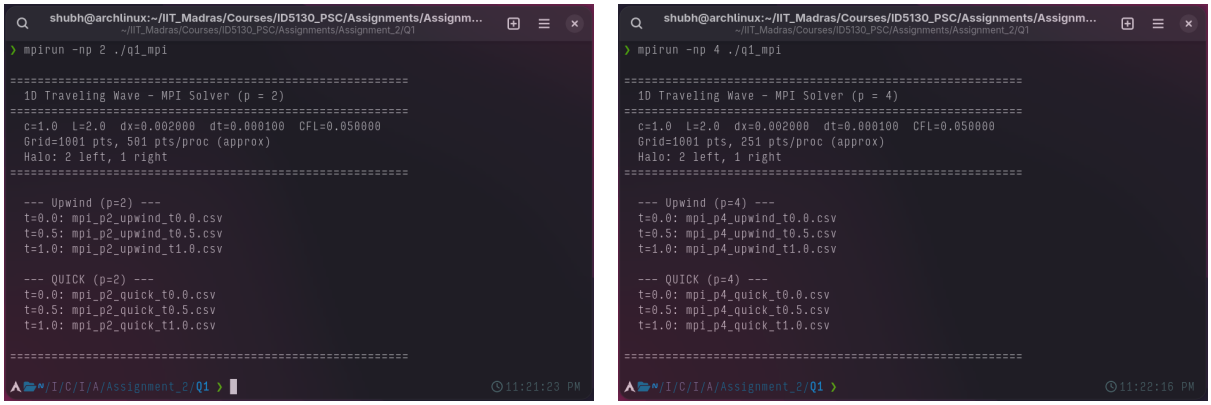
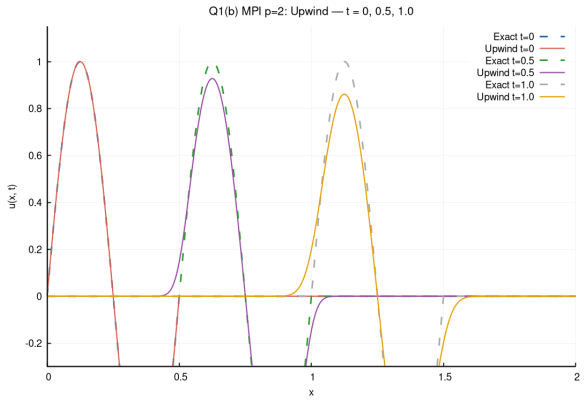
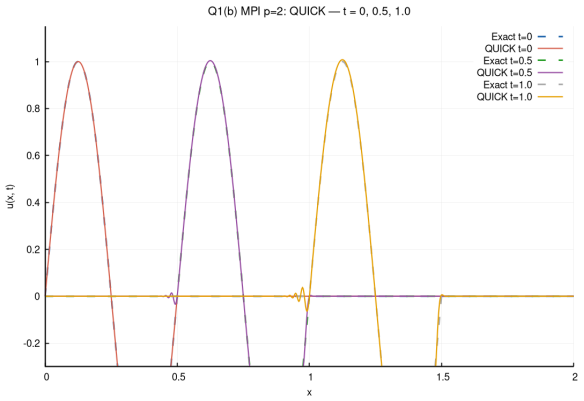


Figure 3: Q1(b): Terminal output of MPI solver for $p = 2$ and $p = 4$.

1.4.4 Plots — $p = 2$



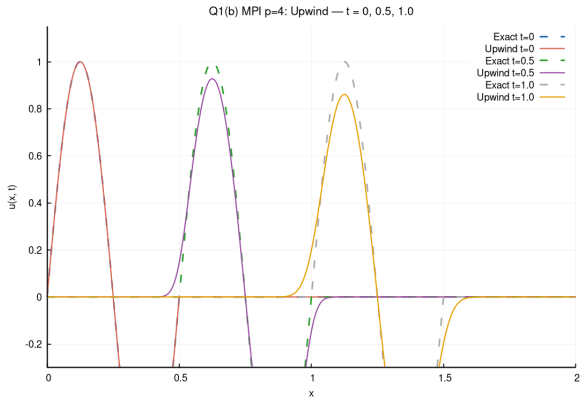
(a) Upwind scheme



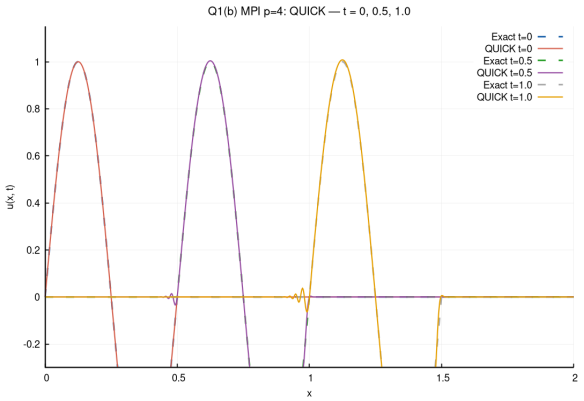
(b) QUICK scheme

Figure 4: MPI $p = 2$ Numerical and exact solutions at $t = 0$, 0.5, 1.0.

1.4.5 Plots — $p = 4$



(a) Upwind scheme



(b) QUICK scheme

Figure 5: MPI $p = 4$ Numerical and exact solutions at $t = 0$, 0.5, 1.0.

Part (c): Comparison of Upwind and QUICK Schemes

1.5.1 Zoomed Comparison at $t = 1.0$

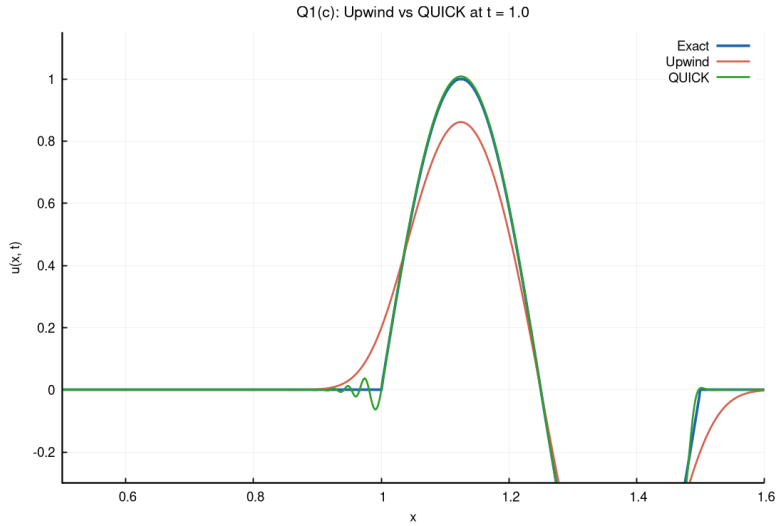


Figure 6: Upwind vs QUICK at $t = 1.0$ (zoomed to the wave region).

1.5.2 Analysis

- **Upwind scheme** exhibits significant **numerical diffusion**. By $t = 1.0$, the peak amplitude drops from 1.0 to approximately 0.85, and the wave profile is visibly smeared and broadened. This is because the leading truncation error of the first-order upwind scheme is proportional to $\Delta x \cdot (\partial^2 u/\partial x^2)$, which acts like an artificial viscosity that smooths sharp gradients.

- **QUICK scheme** preserves the **wave amplitude** much better (peak $\approx$ 1.0 at $t = 1.0$) because it is third-order accurate, with a much smaller truncation error. However, it introduces small **dispersive oscillations** (Gibbs-like wiggles) near the sharp edges of the wave where $u_0(x)$ transitions discontinuously between $\sin(4\pi x)$ and 0. These oscillations are characteristic of higher-order non-dissipative schemes near discontinuities.

- **Trade-off:** The upwind scheme is diffusive but oscillation-free. QUICK is much more accurate in smooth regions but produces spurious oscillations near discontinuities. This is a fundamental distinction in computational fluid dynamics between dissipative and dispersive numerical errors.

- Both the serial and MPI programs produce identical results for each scheme, confirming that the parallelization does not introduce any numerical artifacts.

Question 2: Poisson Equation — Jacobi Method (60 points)

Problem Statement

Solve the Poisson equation:

$$\nabla^2 \phi = -q, \quad q = x^2 + y^2 \tag{7}$$

in the domain $[-1, 1] \times [-1, 1]$ with the following boundary conditions:

$$\phi(x, -1) = 0 \qquad \text{(bottom: Dirichlet)} \tag{8}$$
$$\phi(x, +1) = 0 \qquad \text{(top: Dirichlet)} \tag{9}$$
$$\phi(-1, y) = \sin(2\pi y) \qquad \text{(left: Dirichlet)} \tag{10}$$
$$\left.\frac{\partial \phi}{\partial x}\right|_{x=1} = 0 \qquad \text{(right: Neumann)} \tag{11}$$

Discretization

The domain is discretized with uniform spacing $\Delta = \Delta x = \Delta y$ such that $x_i = -1 + i\Delta$ and $y_j = -1 + j\Delta$, for $i, j = 0, 1, \ldots, N$ where $N = 2/\Delta$.

Jacobi update for interior points ($i = 1, \ldots, N-1;\ j = 1, \ldots, N-1$):

$$\phi_{i,j}^{(k+1)} = \frac{1}{4}\left[\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k)}\right] + \frac{\Delta^2}{4} q_{i,j} \tag{12}$$

Neumann boundary condition at $x = 1$ ($i = N$): The zero-gradient condition $\partial\phi/\partial x = 0$ is approximated using the second-order one-sided formula specified in the problem:

$$\phi_i = \frac{4\phi_{i-1} - \phi_{i-2}}{3} \tag{13}$$

This is applied at every iteration after the interior update, using the newly computed values at $i = N-1$ and $i = N-2$. The formula is derived from Taylor expansion:

$$\phi_{i-1} = \phi_i - \Delta\phi_i' + \frac{\Delta^2}{2}\phi_i'' + \mathcal{O}(\Delta^3)$$
$$\phi_{i-2} = \phi_i - 2\Delta\phi_i' + 2\Delta^2\phi_i'' + \mathcal{O}(\Delta^3)$$

Setting $\phi_i' = 0$ and solving for ϕ_i from $4\phi_{i-1} - \phi_{i-2}$ eliminates the ϕ_i'' term, yielding a second-order accurate approximation.

Convergence criterion: $\max_{i,j} |\phi_{i,j}^{(k+1)} - \phi_{i,j}^{(k)}| < 10^{-4}$.

Initial guess: $\phi(x, y) = 0$ everywhere.

Part (a): Serial Jacobi Program ($\Delta = 0.1$)

2.3.1 Approach

The serial program stores ϕ as a contiguous 1D array of size $(N+1)^2$ indexed as $\phi[i \cdot (N+1) + j]$, where i is the x-index and j is the y-index. Each Jacobi iteration:

1. Updates all interior points ($i = 1 \ldots N-1,\ j = 1 \ldots N-1$) using Eq. (12).
2. Applies the Neumann boundary at $i = N$ using Eq. (13) for $j = 1 \ldots N-1$.
3. Computes the max-norm of the difference between successive iterates.
4. Swaps the old and new arrays via pointer swap.

The Dirichlet boundary at $x = -1$ (left) is set once as $\phi(i = 0, j) = \sin(2\pi y_j)$ and remains fixed throughout the iteration.

2.3.2 Terminal Output

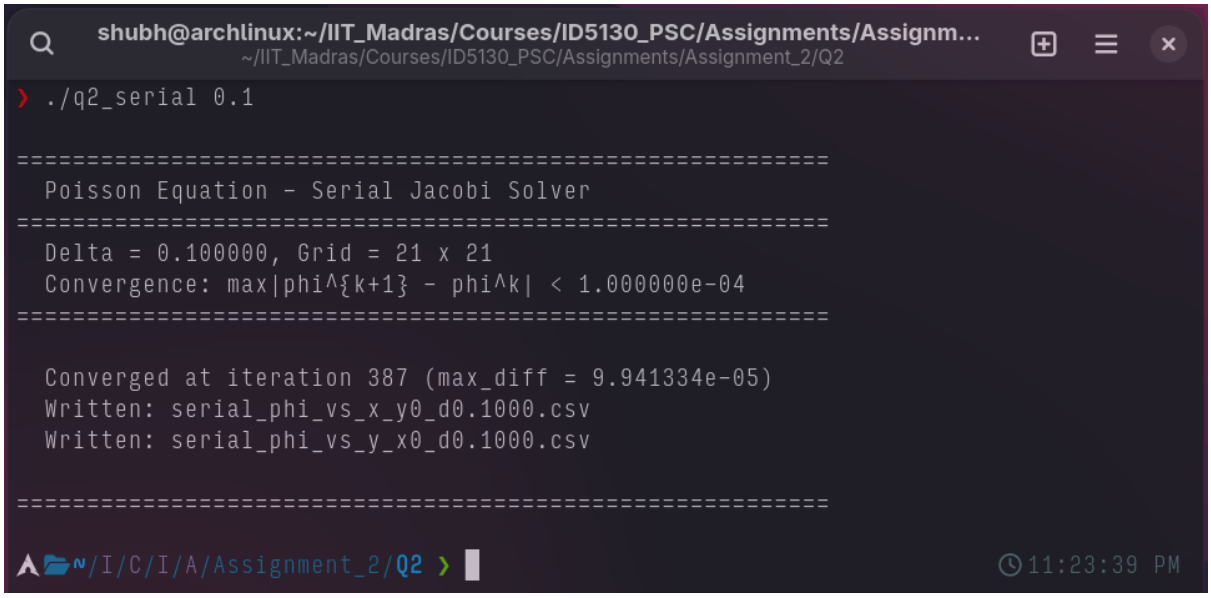
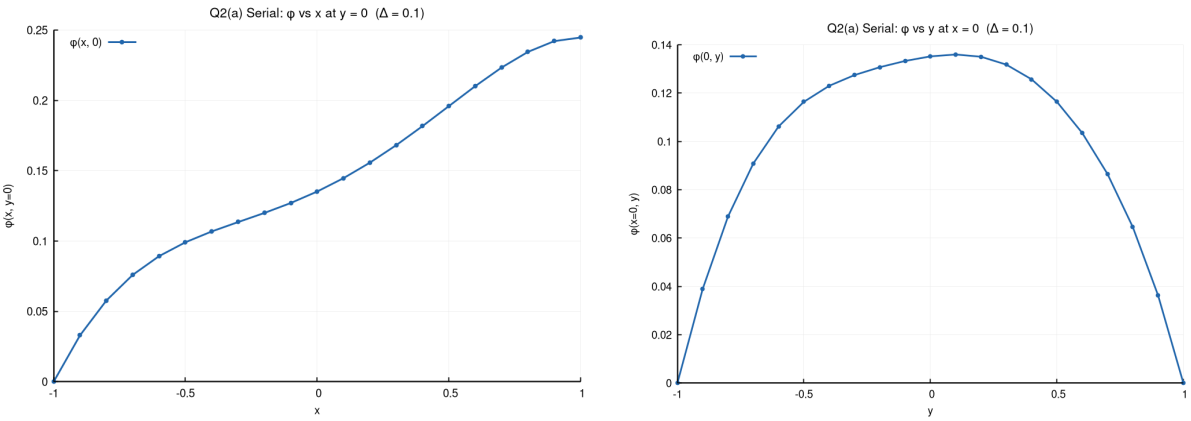


Figure 7: Q2(a): Terminal output of serial Jacobi solver with $\Delta = 0.1$.

2.3.3 Results

- **Grid:** 21×21 ($\Delta = 0.1$)
- **Iterations to converge:** 387
- **Convergence criterion:** $\max |\phi^{k+1} - \phi^k| < 10^{-4}$

2.3.4 Plots



(a) ϕ vs x at $y = 0$ (b) ϕ vs y at $x = 0$

Figure 8: Variation of ϕ for $\Delta = 0.1$.

2.3.5 Key Findings

- The ϕ vs x profile at $y = 0$ shows a local maximum near $x \approx -0.7$ due to the sinusoidal left boundary condition, followed by a monotonic rise toward the right boundary where the Neumann condition ($\partial\phi/\partial x = 0$) forces a zero-slope termination.
- The ϕ vs y profile at $x = 0$ is a smooth, nearly symmetric dome peaking near $y \approx 0.1$, influenced primarily by the source term $q = x^2 + y^2$ and the zero Dirichlet conditions at $y = \pm 1$.
- The solution is physically consistent: the source $q = x^2 + y^2$ drives ϕ positive everywhere in the interior, with the sinusoidal left boundary providing a mild modulation.

Part (b): MPI Jacobi Program ($\Delta = 0.01$)

2.4.1 Approach

The MPI program uses **row-wise 1D domain decomposition** along the x-direction. Each process owns a contiguous block of rows with 1 halo row on each side for the Jacobi stencil.

- **Data layout:** Each process stores $(n_{\text{local}} + 2)$ rows of N_p columns. Row 0 is the left halo, rows $1 \dots n_{\text{local}}$ are owned data, and row $n_{\text{local}} + 1$ is the right halo.
- **Halo exchange:** At each iteration, entire rows (N_p doubles) are exchanged with left and right neighbors via `MPI_Sendrecv`.
- **Neumann BC:** The process owning the rightmost rows ($i = N$) applies Eq. (13) using its local data at $i = N - 1$ and $i = N - 2$. If $N - 2$ falls in the halo, the halo exchange has already provided the correct values.
- **Convergence:** Each process computes its local max $|\phi^{k+1} - \phi^k|$, then `MPI_Allreduce` with `MPI_MAX` obtains the global maximum across all processes.
- **Output:** The full solution is gathered on rank 0 using `MPI_Gatherv`.

2.4.2 Terminal Output

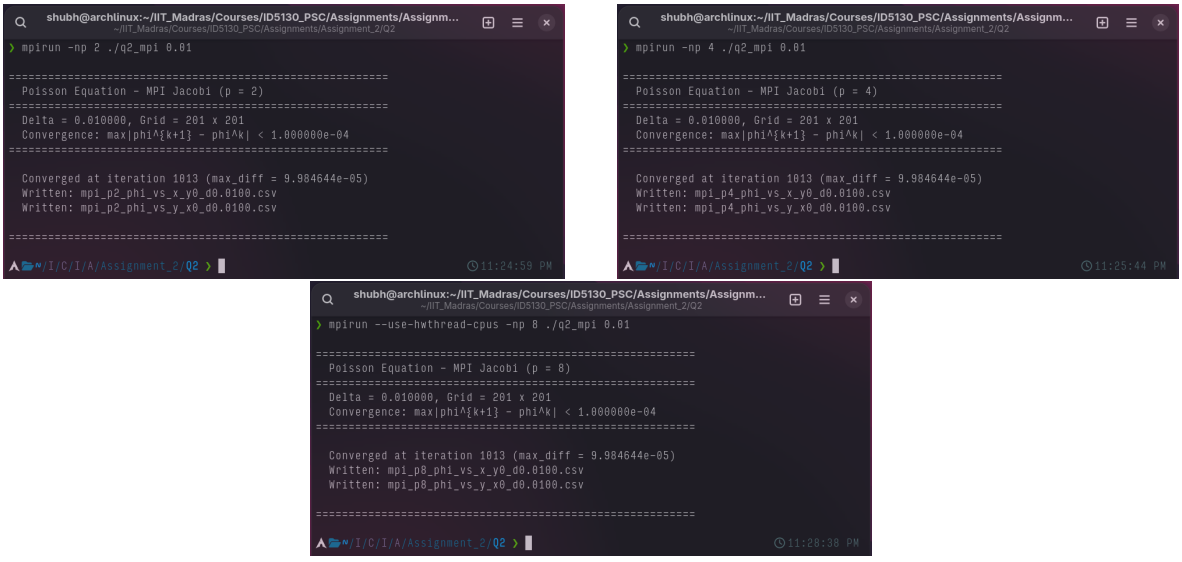


Figure 9: Q2(b): Terminal output of MPI Jacobi solver for $p = 2, 4, 8$ with $\Delta = 0.01$.

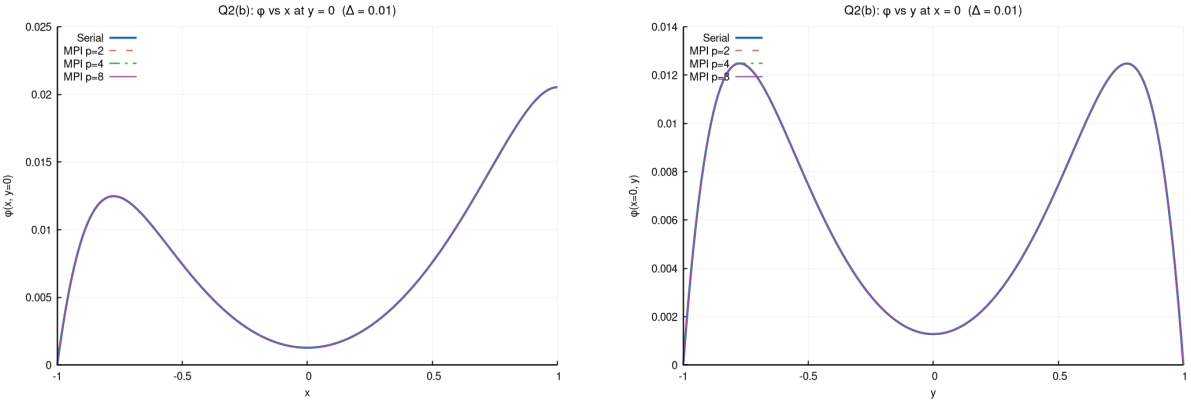
2.4.3 Results

Table 1: Q2(b): Jacobi convergence results for $\Delta = 0.01$ (201×201 grid)

Program	**Iterations**	**Final** $\max\|\Delta\phi\|$	**Match Serial?**
Serial	1013	9.98×10^{-5}	—
MPI $p = 2$	1013	9.98×10^{-5}	Yes (diff = 0.0)
MPI $p = 4$	1013	9.98×10^{-5}	Yes (diff = 0.0)
MPI $p = 8$	1013	9.98×10^{-5}	Yes (diff = 0.0)

All MPI runs converge at exactly the same iteration count as the serial program and produce identical solutions.

2.4.4 Plots



(a) ϕ vs x at $y = 0$ (b) ϕ vs y at $x = 0$

Figure 10: Serial and MPI ($p = 2, 4, 8$) results for $\Delta = 0.01$.

2.4.5 Key Findings

- All four curves (serial, $p = 2$, $p = 4$, $p = 8$) overlap perfectly, confirming that the MPI parallelization with row-wise decomposition and halo exchange is correct and produces results identical to the serial program.
- The Jacobi method converges in 1013 iterations for the 201×201 grid, compared to 387 iterations for the 21×21 grid. The iteration count increases roughly proportionally to N (not N^2), which is characteristic of Jacobi's spectral radius approaching 1 as $\cos(\pi/N)$.
- The Neumann boundary condition at $x = 1$ is correctly handled in the MPI program. The process owning the rightmost rows applies the one-sided formula (Eq. 13), and the zero-slope behavior is visible in the ϕ vs x plot where the curve flattens at $x = 1$.

- The solution profile at $y = 0$ shows the effect of the sinusoidal left boundary ($\phi = \sin(2\pi y) = 0$ at $y = 0$) creating a local hump near $x \approx -0.7$, while the source term $q = x^2 + y^2$ drives the solution to increase toward the right side of the domain.

Reference Material Used

1. **Lecture Slides** – ID5130: Parallel Scientific Computing, Jan–May 2026, IIT Madras.
2. **OpenMP and MPI Documentation** – OpenMP Architecture Review Board, https://www.openmp.org; Open MPI, https://www.open-mpi.org.
3. **An Introduction to Parallel Programming** – Peter Pacheco, Morgan Kaufmann, 2011. ISBN: 978-0-12-374260-5.
4. **gnuplot** – T. Williams, C. Kelley, et al. *gnuplot – An Interactive Plotting Program*, http://www.gnuplot.info. Used for generating graphs and data visualization.